

252-0027

Introduction to Programming Exercises

Week 4: Types, Loops, methods

Jan Leonard

**Department of Computer Science
ETH Zurich**

Organization

My name: *Jan Leonard*

For questions: *leonardj@ethz.ch*

Please put «[EProg26]» in the subject

New exercises: **Tuesday evening** (normally)

Submit exercises by **Tuesday evening (23:59)** following week

Always submit via Git

Solutions in a separate Git project



Eprog TA — HS 2026

janleonard115.github.io

Organization

- This year we switch to a Moodle based Exam
- That's why we publish Moodle Quizzes
- Theory Exercises, Programming Exercises and Moodle Quizzes can be a lot

Organization

- If you submit your Solutions onto GitLab
- I will give you Feedback
- Look at your Exercise Folder for Feedback.md
- If there are still any Questions please email me

x2ygemisch - Task

3. Create a description `<x2ygemisch>` that allows, as legal symbols, exactly those words in which two “Y” occur as a pair for every “X”. Examples are `XYX`, `YYX`, `XYYYXX`, `XXYYYY`. The description does not exclude the empty string.

- Often the submissions looked like this
- `<x2ygemisch> <- [X <x2ygemisch> YY] | [YY<x2ygemisch>X]`
- (Much harder than task 5 - will be changed)

Types

Primitive Typen in Java

Eight primitive types: for numbers, characters, and truth values. Examples:

	<u>Name</u>	<u>Description</u>	<u>Examples</u>
int	whole numbers	-2147483648, -3, 0, 42, 2147483647	
long	large whole numbers	42, -3, 0, 9223372036854775807	
double	real numbers	3.1, -0.25, 9.4e3	
char	(single) characters	'a', 'X', '?', '\n'	
boolean	truth values	true, false	

Compatibility of Selected Primitive Types

Source type (expression)

	Target type (variable)		
	int	long	double
int			
long			
double			



Explicit conversion



Implicit conversion



Implicit conversion – potentially with loss of precision

Examples

- `long x = 1024`
- `int y = (int) x`
- `double d = 9507129656401991L`

→ implicit conversion from **int** to **long**

→ explicit cast required

→ Loss of precision: ...992

Overflow and Underflow

- **For int**

- If we are at Integer.MIN_VALUE or Integer.MAX_VALUE and subtract or add 1 we then wrap around and get Integer.MAX_VALUE or Integer.MIN_VALUE
- No error

- **For double**

- Overflow: If the result is too large it becomes infinity or -infinity
- Underflow: If the result is too close to 0 it becomes 0.0
- No error

Overflow and Underflow

```
int i = Integer.MAX_VALUE;  
System.out.println(i + 1);           // -2147483648  
  
double d = Double.MAX_VALUE;  
System.out.println(d * 2);           // Infinity  
System.out.println(Double.MIN_VALUE / 2); // 0.0
```

Questions: What do these expressions evaluate to?

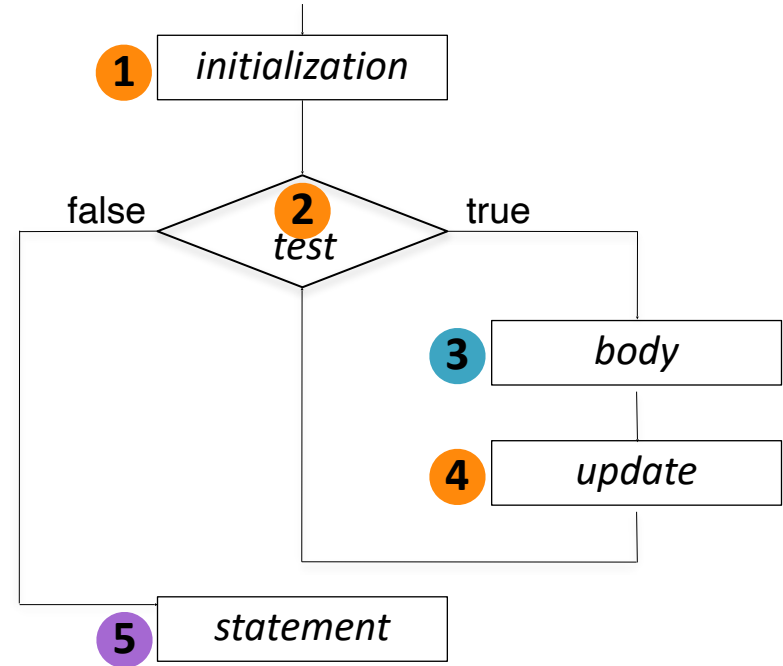
- | | |
|---|------------|
| 1. $(30 * 2 + 5) \% 7 / 3$ | Output: 0 |
| 2. $485 \% (12 + 6 * 2) - 4 * 5 / 2$ | Output: -5 |
| 3. $(8 + 15 * 3 \% 7) / 2 + 9$ | Output: 14 |
| 4. $10 * (6 + 5 \% 4) - 20 / 4 + 8$ | Output: 73 |
| 5. $100 \% (50 / 2) + 75 \% (12 * 2) - 2$ | Output: 1 |
| 6. $(9 + 3) * 2 - 18 \% (5 + 2 * 3)$ | Output: 17 |
| 7. $(6 * 3 + 2) \% 7 - (14 / 2 + 4)$ | Output: -5 |

Loops

for loop: control flow

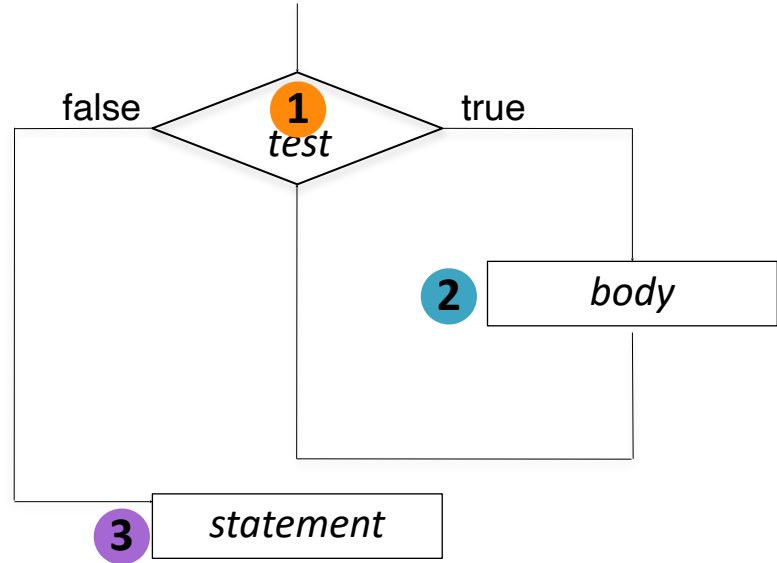
```
for (1initialization; 2test; 4update) {  
    3body  
}  
5statement;
```

```
for (int i = 1; i <= 5; i = i + 1) {  
    System.out.println(i); // 1 2 3 4 5  
}
```



while loop

```
while (test) {  
    body  
}  
statement;
```



while loop executes the body as long as test evaluates to true.
In other words: the while loop stops as soon as test evaluates to false.

Loops

What does this method output?

```

public static void main (String[] args) {
    int f = 0; int g = 1;
    for (int i = 0; i < 15; i++ ) {
        System.out.print(" " + f);
        f = f + g;
        g = f - g;
    }
    System.out.println();
}

```

i	f	g
0	0	1
0	0	1
0	0	1
0	1	1
0	1	0
1	1	0
...	...	...

0 1 1 2 3 5 8 13 21 34 55 89 144 233 377

Loops: While vs. For vs. Do-While

For loop: We already know the number of iterations

While: We do not know the number of iterations

Do-While: We first want to execute the code in the loop body and only then check the condition. Loop body is executed at least once.

Loops: For vs. While

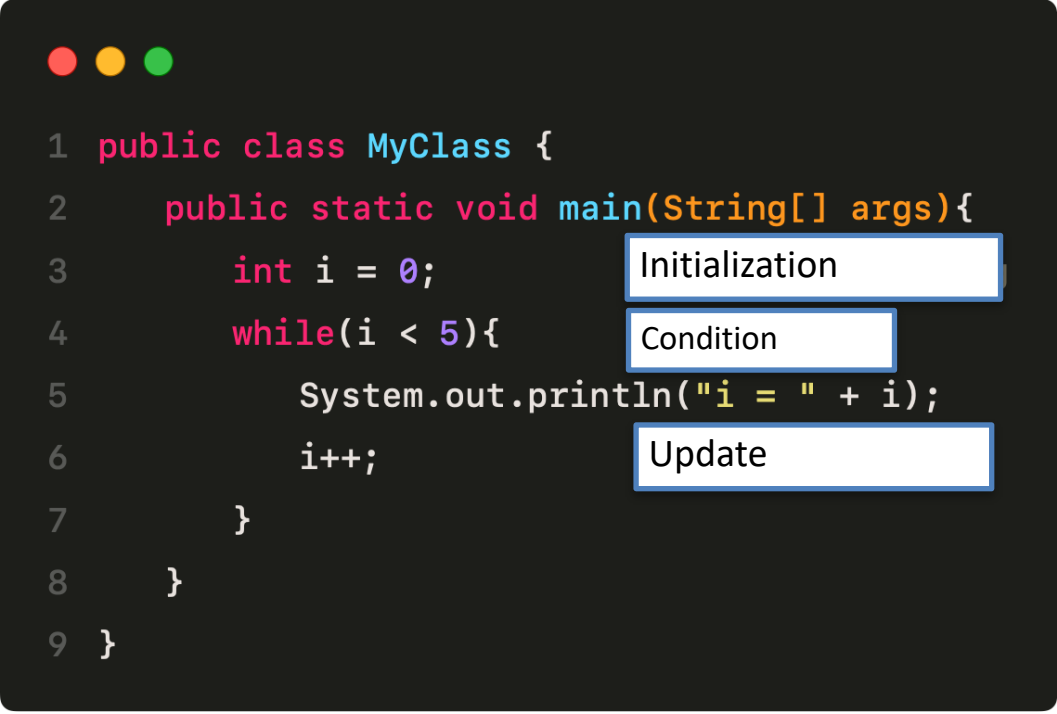
**While and Do-While can do the same thing.
For and While can too!**

Loops: For vs. While – Example 1



```
1 public class MyClass {  
2     public static void main(String[] args){  
3         for(int i = 0; i < 5; i++){  
4             System.out.println("i = " + i);  
5         }  
6     }  
7 }
```

Loops: For vs. While – Example 1



```
1 public class MyClass {  
2     public static void main(String[] args){  
3         int i = 0;           Initialization  
4         while(i < 5){        Condition  
5             System.out.println("i = " + i);  
6             i++;             Update  
7         }  
8     }  
9 }
```

Loops: For vs. While – Example 2



```
1 public class MyClass {  
2     public static void main(String[] args){  
3         int i = 1;  
4         while(fancyCheck(i)){  
5             int result = complexOperation(i);  
6             i = i * 42 - 15;  
7             System.out.println(result);  
8         }  
9     }  
10 }
```

Loops: For vs. While – Example 2



```
1 public class MyClass {  
2     public static void main(String[] args){  
3         int i = 1;  
4         for(; fancyCheck(i); ){  
5             int result = complexOperation(i);  
6             i = i * 42 - 15;  
7             System.out.println(result);  
8         }  
9     }  
10 }
```

Loops: Do-While?

Rarely used, but can be very useful!



```
1 public class MyClass {
2     public static void main(String[] args){
3         Scanner console = new Scanner(System.in);
4         String pin;
5         do {
6             pin = console.nextLine();
7         } while(!isPassword(pin));
8         System.out.println("You logged in successfully!");
9         console.close();
10    }
11 }
```

Increment and Decrement

Increment and Decrement

Prefix: ++i

```
int a = 5;  
int b = ++a; // First a is incremented, then a is assigned.  
// a = 6, b = 6
```

Postfix i++

```
int a = 5;  
int b = a++; // First a is assigned, then a is incremented.  
// a = 6, b = 5
```

These operators are especially useful in loops, for example to increase or decrease a counter.

Increment and Decrement

```
for (int i = 0; i < 5; i++) { ... }  
for (int i = 0; i < 5; ++i) { ... }
```

Other Short Forms

Pre-increment and pre-decrement:

change first, then use

`++variable` for `variable = variable + 1`

`--variable` for `variable = variable - 1`

Change by any value (instead of just 1)

`variable += value` for `variable = variable + value;`

`variable -= value` for `variable = variable - value;`

`variable *= value` for `variable = variable * value;`

`variable /= value` for `variable = variable / value;`

`variable %= value` for `variable = variable % value;`

Be careful:

`x += 3`

`x =+ 3`

Advice for the Exam

- Only use the short forms you are really comfortable with
 - In the exam you will be under stress and time pressure
 - $x =+ 3$ and $x += 3$ can be easily confused
-
- My advice: Practice under real conditions
 - Computer-Rooms in Main Building similar to the exam rooms
 - Simulate the exam or the timed bonus with friends

methods

methods: Sequences and Series

Write a program “Reihe.java” that reads a number N from the console and then computes the following sum:

$$\frac{1}{1^2} + \frac{1}{2^2} + \cdots + \frac{1}{N^2}$$

Example: For $N = 4$, your program should output approx. 1.42. How does this sum behave for large N ?

String

String methods that return String

Method name	Method description
substring(i1, i2) or substring(i1)	A new string: the substring from i1 (inclusive) to i2 (<u>exclusive</u>); if no i2 is given: up to the end of the string
toLowerCase()	A new string with only lowercase letters
toUpperCase()	A new string with only uppercase letters
stripLeading()	A new string without whitespace at the beginning
stripTrailing()	A new string without whitespace at the end

Substrings

```
String name = "Bob Dylan";  
String firstName = name.substring(0, 3); // "Bob"  
String lastName = name.substring(4); // "Dylan"  
String firstCharacter = name.substring(0, 1); // "B"  
name = name.toLowerCase(); // "bob dylan"
```

Indices 0, 1, 2

From index 4
until the end

name:	B	o	b		D	y	l	a	n
Index	0	1	2	3	4	5	6	7	8

Careful: `name.substring(0, 1)` is not the same as `name.charAt(0)`

`name.substring(0, 1)` has value "B" and is of type String
`name.charAt(0)` has value 'B' and is of type char

String methods that return int

Method name	Method description
length()	Length of the string (number of characters)
indexOf(s, fromIndex) indexOf(c, fromIndex) indexOf(s) indexOf(c)	Index where string s or character c first occurs in the string, starting the search at index fromIndex (-1 if not found); if fromIndex is not given, the search starts at the beginning of the string

```
String s = "S. Beckett";  
int indexBeck = s.indexOf("Beck");    // 3  
int eFirstOccurrence = s.indexOf('e'); // 4  
int eSecondOccurrence = s.indexOf('e', 5); // 7
```

String methods that return boolean

Method name	Method description when calling s1.method(s2)
<code>equals(s2)</code>	whether s1 and s2 contain the same characters in the same order
<code>equalsIgnoreCase(s2)</code>	whether s1 and s2 contain the same characters in the same order, ignoring upper and lower case
<code>startsWith(s2)</code>	whether s1 starts with the characters of s2
<code>endsWith(s2)</code>	whether s1 ends with the characters of s2
<code>contains(s2)</code>	whether s1 contains s2 anywhere

String Comparison

- This is just a preview
- Strings are **Objects** and **immutable**
- Use `==` only on **primitive Types** in Java
- Using `==` on Strings in EProg counts as **wrong**
- Always use `.equals()` to compare Strings
- Also important for the **Theory Exam**

Where can you find this information?

In the Java Documentation 25



The screenshot shows the top of the Java SE 25 & JDK 25 API Specification page. It includes a search bar, the title "Java® Platform, Standard Edition & Java Development Kit Version 25 API Specification", and a section titled "This document is divided into two sections:". Below this, there are two sections: "Java SE" and "JDK". The "Java SE" section describes the core Java platform APIs. The "JDK" section describes the Java Development Kit APIs. At the bottom, there is a table with two columns: "Module" and "Description". The table lists two modules: "java.base" and "java.compiler".

Java SE 25 & JDK 25

Search documentation (type)

Java® Platform, Standard Edition & Java Development Kit Version 25 API Specification

This document is divided into two sections:

Java SE

The Java Platform, Standard Edition (Java SE) APIs define the core Java platform for general-purpose computing. These APIs are in modules whose names start with java.

JDK

The Java Development Kit (JDK) APIs are specific to the JDK and will not necessarily be available in all implementations of the Java SE Platform. These APIs are in modules whose names start with jdk.

All Modules **Java SE** **JDK** **Other Modules**

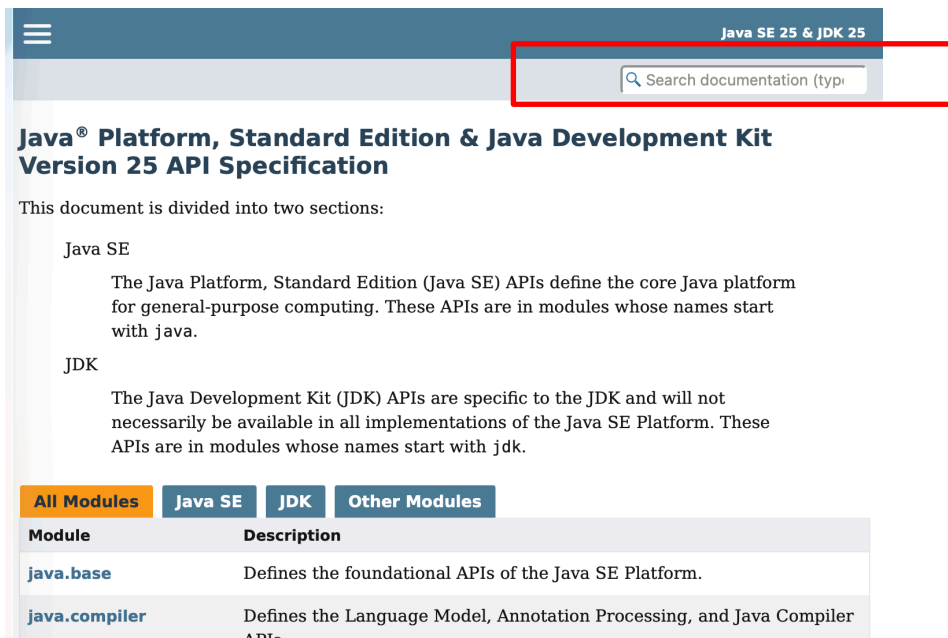
Module	Description
java.base	Defines the foundational APIs of the Java SE Platform.
java.compiler	Defines the Language Model, Annotation Processing, and Java Compiler APIs.



<https://docs.oracle.com/en/java/javase/25/docs/api/>

Where can you find this information?

In the Java Documentation 25



The screenshot shows the top of the Java SE 25 & JDK 25 API Specification page. A red rectangle highlights the search bar in the top navigation bar, which contains the text "Search documentation (type)". Below the navigation bar, the page title "Java® Platform, Standard Edition & Java Development Kit Version 25 API Specification" is visible. The main content area is divided into two sections: "Java SE" and "JDK". The "Java SE" section describes the core Java platform APIs. The "JDK" section describes APIs specific to the JDK. At the bottom, there is a table with two columns: "Module" and "Description". The table lists two modules: "java.base" and "java.compiler".

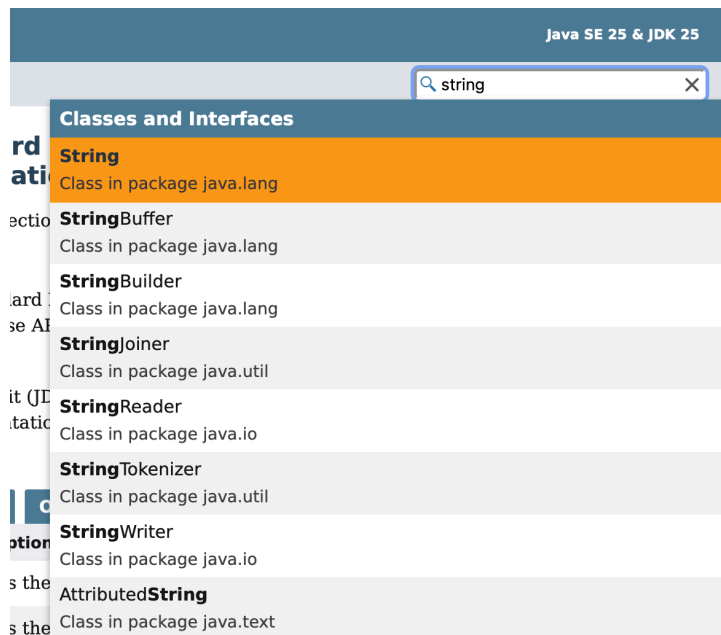
Module	Description
java.base	Defines the foundational APIs of the Java SE Platform.
java.compiler	Defines the Language Model, Annotation Processing, and Java Compiler APIs.



<https://docs.oracle.com/en/java/javase/25/docs/api/>

Where can you find this information?

In the Java Documentation 25



<https://docs.oracle.com/en/java/javase/25/docs/api/>

Where can you find this information?

In the Java Documentation 25



Method Summary

All Methods

Static Methods

Instance Methods

Concrete Methods

Deprecated Methods

Modifier and Type	Method	Description
char	charAt(int index)	Returns the char value at the specified index.
InputStream	chars()	Returns a stream of int zero-extending the char values from this sequence.
int	charAt(int index)	Returns the character (Unicode code point) at the specified index.
int	codePointBefore(int index)	Returns the character (Unicode code point) before the specified index.
int	codePointCount(int beginIndex, int endIndex)	Returns the number of Unicode code points in the specified text range of this String.
InputStream	codePoints()	Returns a stream of code point values from this sequence.
int	compareTo(String anotherString)	Compares two strings lexicographically.
int	compareToIgnoreCase(String str)	Compares two strings lexicographically, ignoring case differences.
String	concat(String str)	Concatenates the specified string to the end of this string.

Click Method Name

<https://docs.oracle.com/en/java/javase/25/docs/api/>

Where can you find this information?

In the Java Documentation 25

charAt

```
public char charAt(int index)
```

Returns the `char` value at the specified index. An index ranges from 0 to `length() - 1`. The first `char` value of the sequence is at index 0, the next at index 1, and so on, as for array indexing.

If the `char` value specified by the index is a [surrogate](#), the surrogate value is returned.

Specified by:

`charAt` in interface [CharSequence](#)

Parameters:

`index` - the index of the `char` value.

Returns:

the `char` value at the specified index of this string. The first `char` value is at index 0.

Throws:

[IndexOutOfBoundsException](#) - if the `index` argument is negative or not less than the length of this string.



<https://docs.oracle.com/en/java/javase/25/docs/api/>

String Examples



```
1 public class MyClass{  
2     public static void main(String[] args){  
3         String str = "Substring";  
4         int result = str.indexOf('S') + str.lastIndexOf('g') - str.length();  
5     }  
6 }
```

Output: -1

String Examples



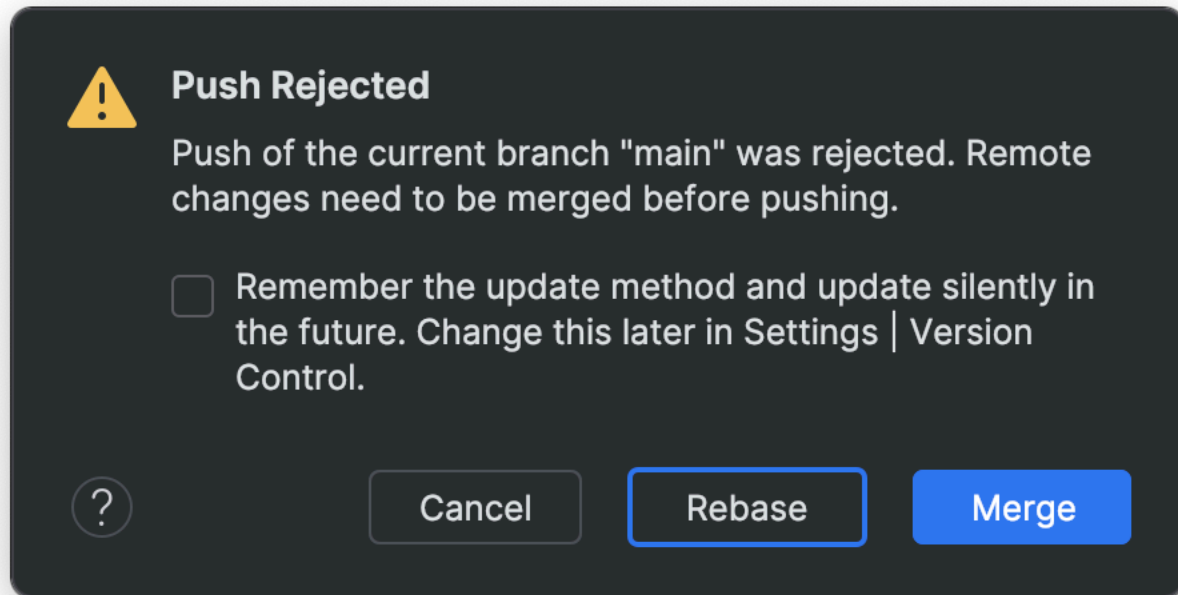
```
1 public class MyClass{  
2     public static void main(String[] args){  
3         String str = "EPROG2024";  
4         int result = str.indexOf('2') + str.length() * 2 - 10;  
5     }  
6 }
```

Output: 13

Kahoot

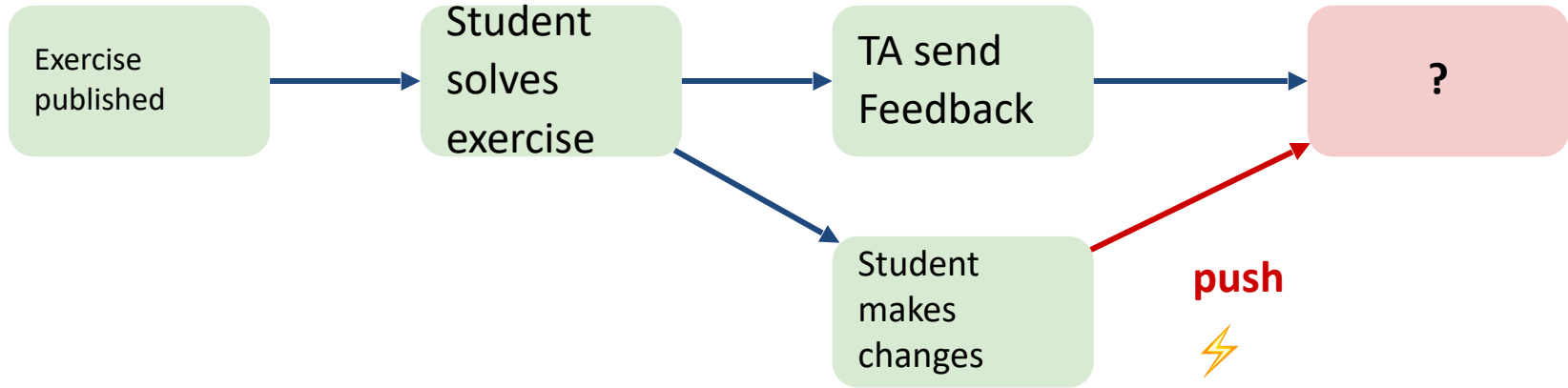
Git Merge Conflicts

Git says "Push Rejected"



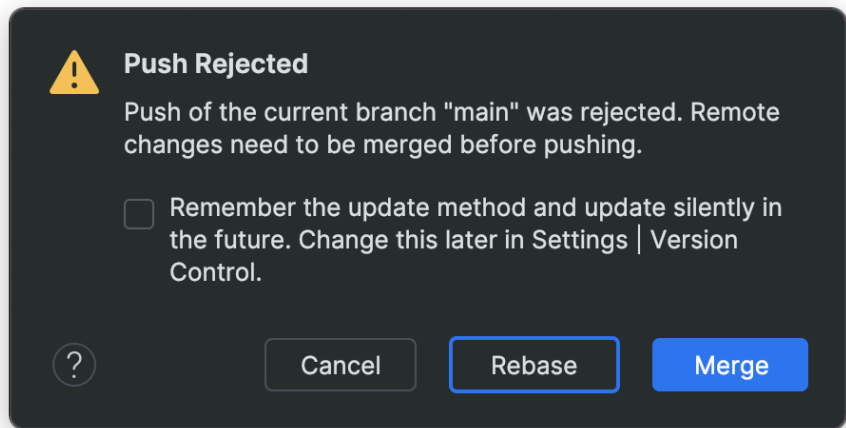
Git - Merge Conflicts

Student solves the exercise while TA is grading
→ Conflict when trying to push!



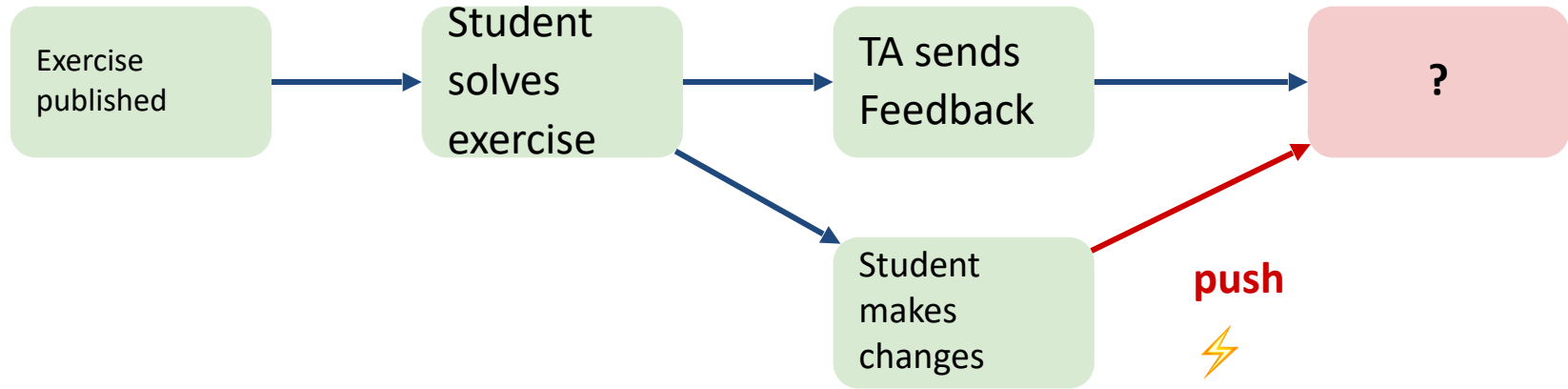
Two Options

- **Merge:** combines the two different versions of the file into one combined file.
- **Rebase:** takes the changes from the server and tries to apply your changes.

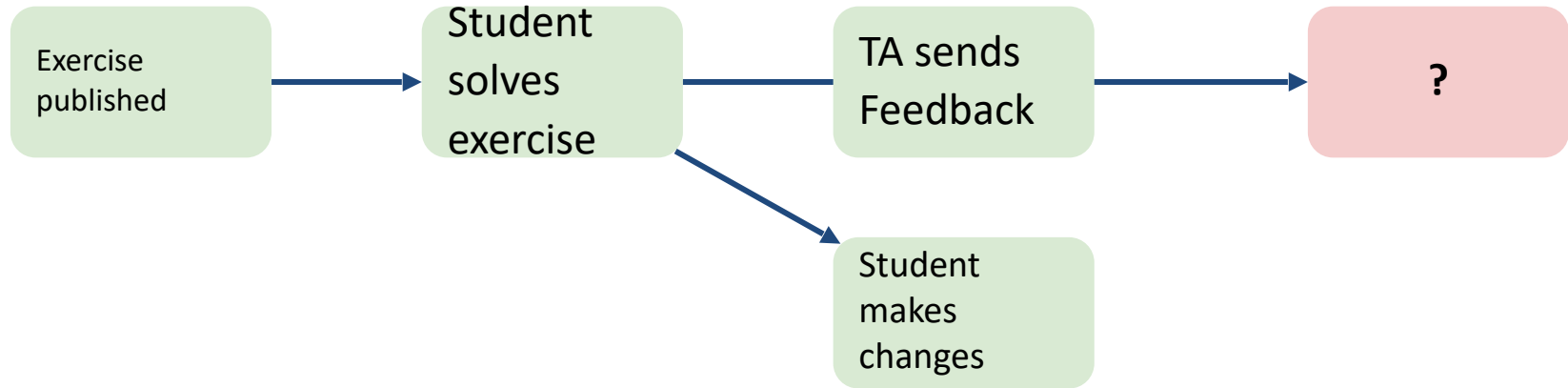


Git Merge

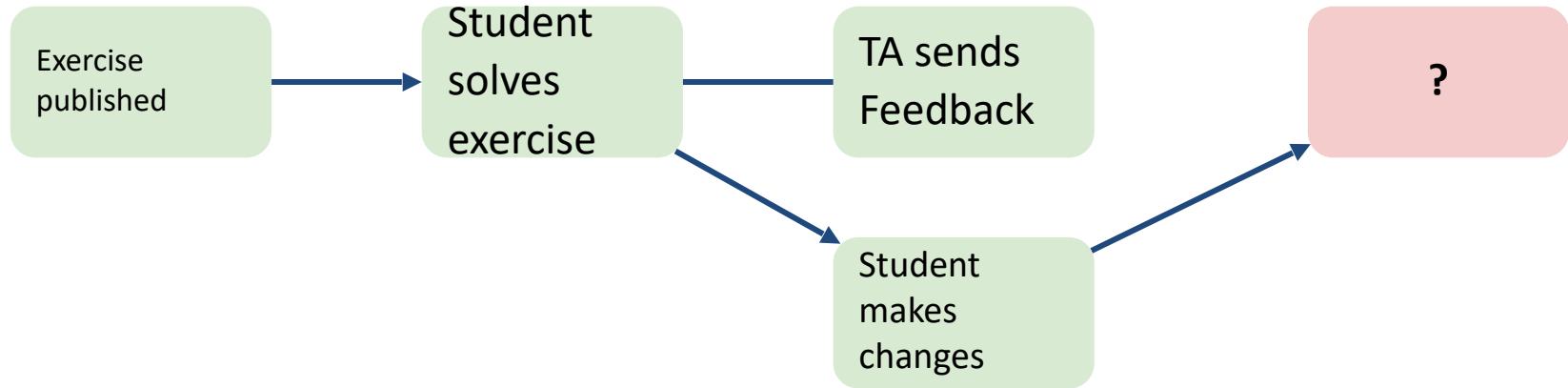
Git – Merge



Git – Merge (Accept Theirs)

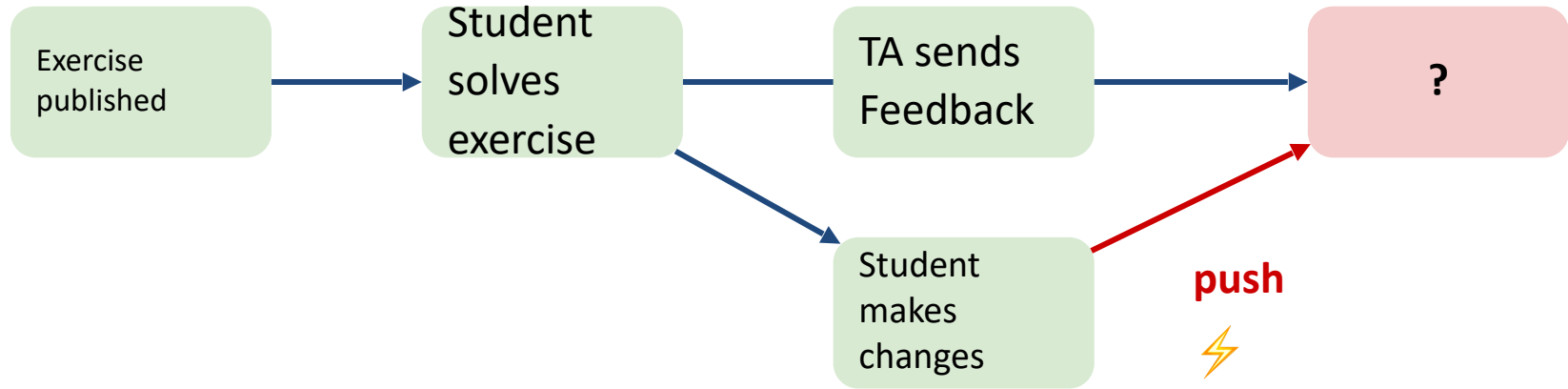


Git – Merge (Accept Yours)

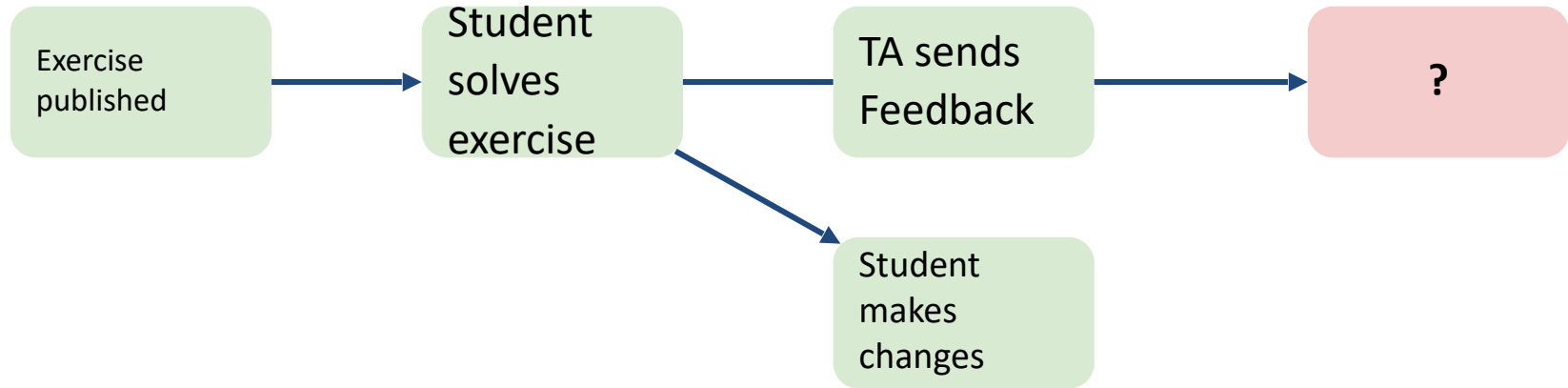


Git Rebase

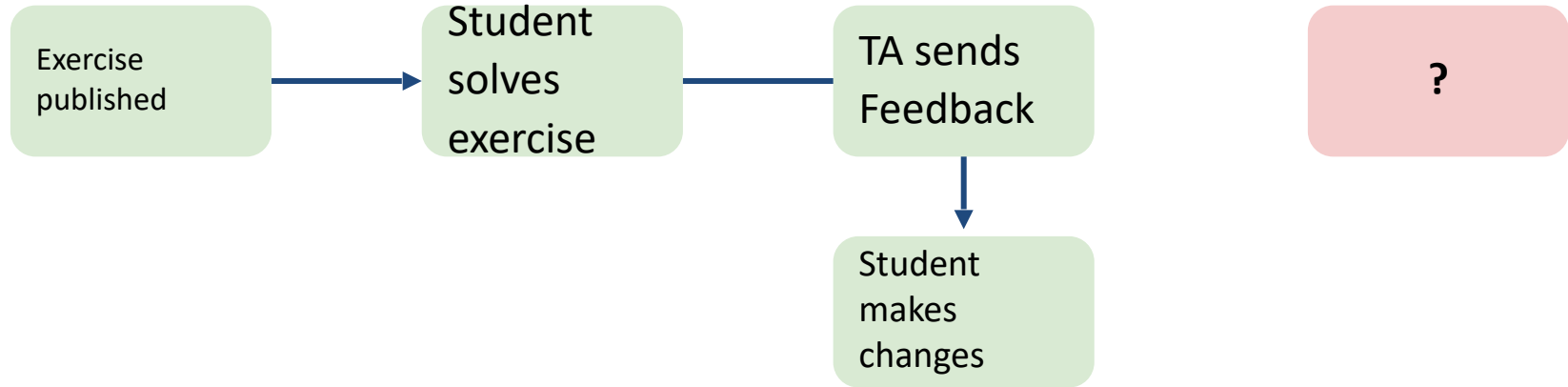
Git – Rebase



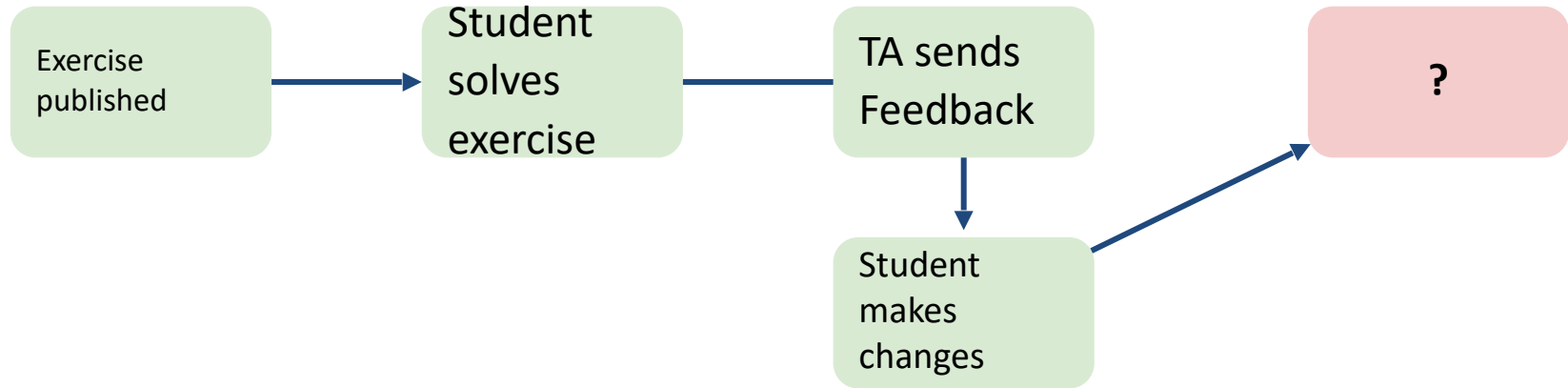
Git – Rebase



Git – Rebase



Git – Rebase

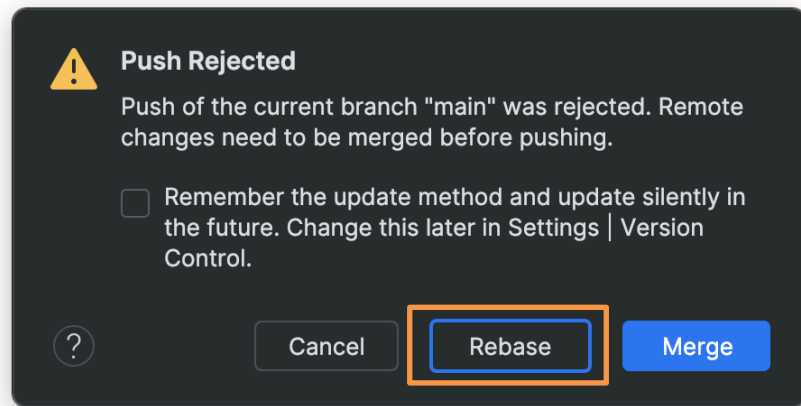


Two Situations

- TA and student did not edit the same file(s). 🎉
- TA and student edited the same file(s). 😓
- (Please do not write in the Feedback file)

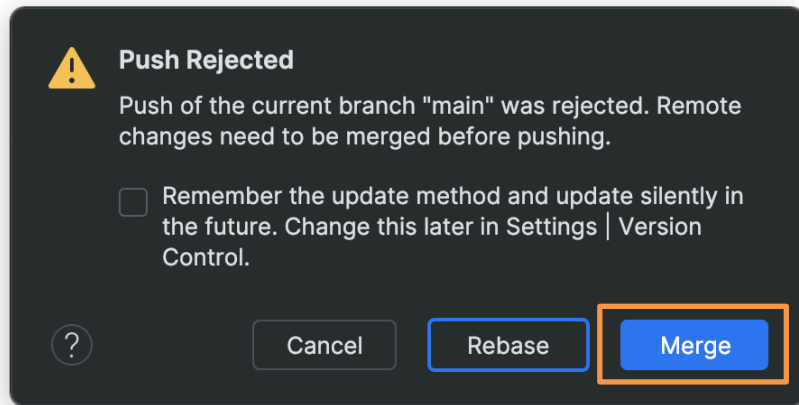
Two Situations

- TA and student did not edit the same file(s). 🥳
- *Choose Rebase.*
- Changes have been pushed.



Two Situations

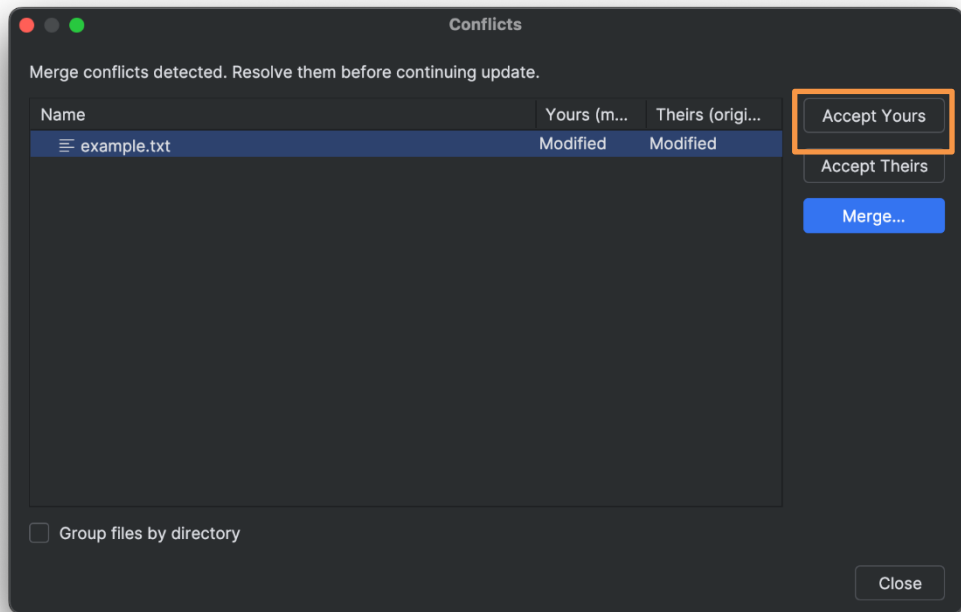
- TA and student edited the same file(s). 🥵
 - *Choose Merge.*



Two Situations

- TA and student edited the same file(s). 🥵

- *Choose Merge.*
- *Choose Accept Yours.*
- Changes pushed.



Preview

Exercise 1: Binary Representation

Write a program “Binaer.java” that reads in a positive number Z and then prints the [binary representation](#).

Hint: First find the largest number k such that the power of two $K = 2^k$ is less than or equal to Z . This is the first 1-bit; after that, you can subtract the power of two and go downward through the powers of two. If $K = 2^{k-1}$ is greater than the remaining number, then the next bit is 1, otherwise 0, etc.

Example: For $Z = 14$, your program should output 1110.
Note that we do not use two’s complement here
and only positive numbers should be represented.

Exercise 2: Greatest Common Divisor

Write a program “GGT.java” that computes the greatest common divisor (GCD) of two integers using the Euclidean algorithm. This is an iterative computation based on the following observation:

1. If x is greater than or equal to y and x can be divided by y without remainder, then the GCD of x and y is equal to y ;
2. otherwise, the GCD of x and y is the same as the GCD of y and $x \% y$.

First practice this algorithm by hand on a few examples and then write the Java program.

Example: For $x = 36$ and $y = 44$:

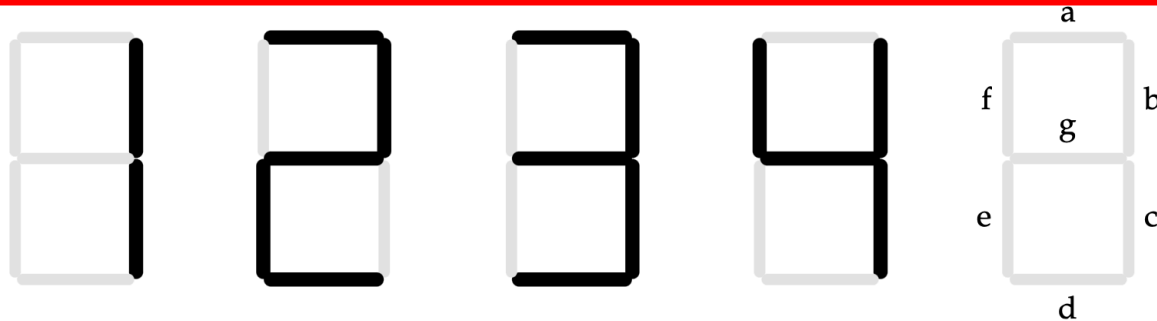
$$ggT(\underbrace{36}_x, \underbrace{44}_y) \stackrel{2.}{=} ggT(44, \underbrace{36 \% 44}_{36}) \stackrel{2.}{=} ggT(36, \underbrace{44 \% 36}_8) \stackrel{2.}{=} ggT(8, \underbrace{36 \% 8}_4) \stackrel{1.}{=} 4$$

Exercise 3: Number Recognition

For this task, we use a string to encode the lit segments of a [seven-segment display](#). The segments are numbered from a to g, as shown in the image. The encoding of a possible display is a string in which the letter x occurs exactly when the x-th segment of the display is lit. For example, the number 2 is encoded by 'abged'. For simplicity, you may assume that no letter occurs more than once in the encoding and that only the numbers 0 to 9 are encoded.

Write a program "Zahlen.java" that reads in a string encoding a display and prints the encoded number as an integer. Think about how many if blocks are needed to recognize every number.

Tip: You can use `str.contains('a')` to check whether a string `str` contains the letter 'a'.



Exercise 4: Calculations

1. Implement the method `Calculations.magic7(int a, int b)`. The method returns a boolean. The method should return `true` if at least one of the following conditions is fulfilled: the first parameter is 7, the second parameter is 7, the sum or one of the differences of the parameters is 7. Otherwise, the method should return `false`. The differences of the parameters `a` and `b` are defined as $a - b$ and $b - a$.

Examples

- `magic7(2,5)` returns `true`.
- `magic7(7,9)` returns `true`.
- `magic7(14,7)` returns `true`.
- `magic7(7,14)` returns `true`.

Exercise 4: Calculations

2. Implement the method `Calculations.fast12(int z)`. The argument `z` is nonnegative. The method returns a boolean. The method should return `true` if `z` is close to a multiple of 12. A number `x` is close to a number `y` if one of the numbers is at most 2 greater or smaller than the other number. Otherwise, the method should return `false`.

- `fast12(12)` returns `true`.
- `fast12(14)` returns `true`.
- `fast12(10)` returns `true`.
- `fast12(15)` returns `false`.

Hint: With the function `Math.abs(num)`, you can obtain the absolute value of a number `num`.

Exercise 5: Scrabble

In this task, you should lay Scrabble tiles using ASCII art on the console. Complete the method `drawNameSquare` in the class `Scrabble`. This method takes a name as a string parameter and should print the name as Scrabble tiles arranged in a square on the console (`System.out`). If, for example, the string `Alfred` is passed, the following image should be output:

```
+---+---+---+---+---+---+
| A | L | F | R | E | D |
+---+---+---+---+---+---+
| L |           | E |
+---+           +---+
| F |           | R |
+---+           +---+
| R |           | F |
+---+           +---+
| E |           | L |
+---+---+---+---+---+---+
| D | E | R | F | L | A |
+---+---+---+---+---+---+
```

Exercise 5: Scrabble

Your code only needs to support names of length 3 or longer. For a name of length 3, e.g. Jim, the output should look like this:

```
+---+---+---+
| J | I | M |
+---+---+---+
| I |   | I |
+---+---+---+
| M | I | J |
+---+---+---+
```

Note that your program must not produce any output other than the Scrabble square.

Exercise 6: Operators - Fundamentals

Task 6: Operators: Fundamentals

In this task, you will examine the order in which Java evaluates the components of an expression. Pay particular attention to operator precedence and associativity, integer division, string concatenation, type conversions, and the short-circuit evaluation of boolean expressions. The following code fragments are independent of one another and are each executed inside a `main` method.

For each subtask, determine the exact console output without running the code. Take into account spaces, line breaks, and the data type of the result. Explicitly mark every space in your answer with `␣`. Whenever the console output starts a new line, start a new line in your answer as well. Afterwards, you may check your answers by running the code.

a) `System.out.println(18 - 3 * 4 + 17 % 5);`

Output: _____

Exercise 6: Operators - Advanced

Task 7: Operators: Advanced

In this task, you will combine several rules for evaluating Java expressions. In addition, pay attention to parentheses, mixed integer and floating-point arithmetic, prefix and postfix increments, the conditional operator, assignments, casts, and the representation of double values. As in Task 6, all code fragments are independent of one another and are each executed inside a `main` method.

Determine the exact console output without running the code. Mark every space with `_` and reproduce every line break from the console output in your answer.

a) `System.out.println(48 / (4 / 2) + 7 % 4);`

Output: _____

Thank you very much for your attention

Have a great week!